

Dynamic Treasure Hunt

Design Document

1. Terminology & Formulae

The expedition: An expedition starts at CAMP. The program randomly selects one hidden treasure location from TEMPLE, CAVE, WATCHTOWER, RUINS, or SWAMP. The treasure remains hidden while the player explores the map.

Routes and challenges: The map is represented as a graph. Each route contains a destination, distance, terrain, and energy cost. A challenge is generated from the currently available routes. The player solves the challenge to choose a route.

Resources: Easy → 42 Energy, 16 Moves, 5 Hints

Medium → 32 Energy, 13 Moves, 4 Hints

Hard → 27 Energy, 11 Moves, 3 Hints

Dynamic map: Some paths are closed at the start. Rain increases Forest and Rough route costs by 2. Rockfall can close a path and path reopening can open a closed path.

Discoveries: The first visit to an eligible discovery location gives 25 points. Important game actions, discoveries, events, and completed expeditions are recorded.

End of the expedition: The expedition ends when the treasure is found, energy is exhausted, moves are exhausted, or no usable route remains. The player can also quit using Q and confirmation.

Formulae

- $\text{Remaining Energy} = \text{Current Energy} - \text{Route Energy Cost}$
- $\text{Energy Used} = \text{Maximum Energy} - \text{Current Energy}$
- $\text{Total Distance} = \text{Current Distance} + \text{Route Distance}$
- $\text{Discovery Score} = \text{Current Score} + 25$
- $\text{Rain Route Cost} = \text{Base Route Cost} + 2$

2. Program Flow

1. Open the main menu.
2. Choose Start New Expedition.
3. Choose Easy, Medium, or Hard.

4. Initialize the game state, map, and hidden treasure.
5. Start the player at CAMP.
6. Display the game-world interface and current expedition information.
7. Find the currently available routes.
8. Generate and display a challenge based on the available routes.
9. Read and process the player's input.
10. If the player requests a hint, map, status, or quit, process the corresponding command.
11. If a route is selected, validate the route against the challenge.
12. If the decision is correct, update the location, energy, moves, distance, and route history.
13. If the decision is wrong, apply the configured penalty and allow another attempt.
14. Check for discoveries and award discovery points when applicable.
15. Check for dynamic events and apply their effects.
16. Check the expedition ending conditions.
17. Continue the expedition until the treasure is found or an ending condition occurs.
18. Display the final result.
19. Save question and expedition history.

3. File Organization & Data Structures

File Organization

File	Responsibility
main.py	Contains the user interface, input handling, menu control, expedition control, display functions, and main game loop.
test.py	Contains automated test cases that verify the correctness of game functions, game rules, map operations, clue generation, resource updates, route handling, edge cases, and expected game behaviour.
logic.py	Contains game rules, map/graph logic, challenge generation and validation, events, discoveries, game-state updates, question history, expedition history, and supporting game data/constants.

Data Structures

- **Dictionary** — Stores the recent five completed game histories in order. It is also used for collections such as routes, answer options, and player path.
- **List** — Stores used clues for each game to prevent the same clue from being repeated.
- **Set** — Stores game state, difficulty settings, map locations, and route information using key-value pairs.
- **Tuple** — Stores fixed route information such as destination, distance, and energy cost.
- **Graph** — The treasure-hunt map is represented as a graph using locations as nodes and routes as connections.
- **Queue** — Used by BFS to check route reachability.

4. Function Signatures

Function	Purpose
<code>enable_terminal_colors()-> None:</code>	Enables ANSI terminal colors on Windows.
<code>clear_screen()-> None:</code>	Clears the terminal screen.
<code>show_startup_screen()-> None:</code>	Displays the game opening screen and title.
<code>color_text(text: str, color: str = WHITE, bold: bool = False) -> str:</code>	Formats text using the selected terminal color and optional bold style.
<code>print_title(title: str) -> None:</code>	Displays a formatted section title.
<code>print_success(text: str) -> None:</code>	Displays a success message.
<code>print_warning(text: str) -> None:</code>	Displays a warning message.
<code>print_error(text: str) -> None:</code>	Displays an error message.
<code>print_info(text: str) -> None:</code>	Displays an informational message.
<code>print_option(number: int, text: str) -> None:</code>	Displays a numbered menu or answer option.
<code>make_route_key(place: str, destination: str) -> tuple[str, str]:</code>	Creates a unique route identifier for two connected locations.
<code>get_all_routes()-> list[tuple[str, str]]:</code>	Returns all unique routes in the map.

Function	Purpose
show_map(game: dict None = None) -> None:	Displays the fixed treasure-hunt map with distances and closed routes.
get_open_routes(location: str, game: dict) -> list[tuple[str, int, int]]:	Returns all currently open routes from a location.
is_reachable(start: str, target: str, game: dict) -> bool:	Checks whether a target location is reachable using open routes.
close_initial_routes(game: dict) -> None:	Randomly closes the required number of routes at game start.
create_game(level: str) -> dict:	Creates and initializes a new game state.
make_direct_clue(game: dict, routes: list[tuple[str, int, int]]) -> dict:	Creates a clue based on a directly reachable location.
make_distance_clue(game: dict, routes: list[tuple[str, int, int]]) -> dict:	Creates a clue based on route distance comparison.
make_map_clue(game: dict, routes: list[tuple[str, int, int]]) -> dict None:	Creates a clue requiring two-step map reasoning.
make_clue_id(clue: dict) -> str:	Creates a unique identifier for a clue.
clue_was_recent(clue: dict, game: dict) -> bool:	Checks whether a clue was already used in the current or recent games.
remember_clue(clue: dict, game: dict) -> None:	Stores the current clue as used.
save_game_questions(game: dict) -> None:	Saves the current game's used clues in the recent-game history.
generate_clue(game: dict) -> dict None:	Generates a new valid clue that has not been recently used.
check_answer(choice: int, clue: dict, game: dict) -> bool:	Validates the player's answer and updates score/resources.
give_hint(clue: dict, game: dict) -> None:	Uses a hint to remove one incorrect answer option.

Function	Purpose
move_player(game: dict, destination: str) -> bool:	Moves the player along an open route and updates game resources.
change_routes(game: dict) -> None:	Dynamically closes or reopens routes during gameplay.
play_game(level: str) -> None:	Controls one complete treasure-hunt expedition.
choose_difficulty()-> str:	Gets and validates the player's difficulty selection.
show_instructions()-> None:	Displays the game rules and available commands.
main()-> None:	Controls the main menu and overall program execution.

5. Function-Level Algorithm

5.1 enable_terminal_colors()

1. Check whether the operating system is Windows.
2. If it is Windows, execute the command that enables ANSI terminal color support.
3. Otherwise, make no changes.

5.2 clear_screen()

1. Check the operating system.
2. Use cls for Windows.
3. Use clear for other operating systems.
4. Execute the selected command.

5.3 show_startup_screen()

1. Display the bordered game title.

2. Display the game subtitle and description.
3. Display the expedition information.
4. Ask the player to press ENTER to continue.

5.4 color_text(text, color, bold)

1. Select the bold style if bold is enabled.
2. Combine the selected style, color, text, and reset code.
3. Return the formatted string.

5.5 print_title(title)

1. Print a separator line.
2. Center and display the supplied title.
3. Print another separator line.

5.6 print_success(text)

1. Format the supplied text using green and bold styling.
2. Print the formatted message.

5.7 print_warning(text)

1. Format the supplied text using yellow and bold styling.
2. Print the formatted message.

5.8 print_error(text)

1. Format the supplied text using red and bold styling.
2. Print the formatted message.

5.9 print_info(text)

1. Format the supplied text using cyan styling.
2. Print the formatted message.

5.10 print_option(number, text)

1. Convert the option number to text.
2. Format the number using the menu color.

3. Display the option number followed by its description.
-

5.11 make_route_key(place, destination)

1. Receive two connected locations.
2. Sort the two location names.
3. Combine them into a tuple.
4. Return the tuple as the unique route key.

5.12 get_all_routes()

1. Create an empty route list.
2. Traverse every location in MAP.
3. Traverse every destination connected to that location.
4. Create a route key using make_route_key().
5. Add the route only if it is not already present.
6. Return the list of unique routes.

5.13 show_map(game=None)

1. Display the map title.
2. Obtain the currently closed routes if a game is provided.
3. Display each location using the appropriate color.
4. Display every road with its distance.
5. Mark closed roads with a red X.
6. Display the map legend and status of the roads.

5.14 get_open_routes(location, game)

1. Create an empty list of routes.
2. Read all routes connected to the given location.
3. Create a route key for each route.
4. Check whether the route is closed.

5. Add only open routes to the result.
6. Return the available routes.

5.15 is_reachable(start, target, game)

1. Add the starting location to the visited set.
 2. Add the starting location to the queue.
 3. Remove a location from the queue.
 4. If it is the target, return True.
 5. Examine its connected locations.
 6. Ignore closed routes and already visited locations.
 7. Add new reachable locations to the queue.
 8. Continue until the target is found or the queue becomes empty.
 9. Return False if the target cannot be reached.
-

5.16 close_initial_routes(game)

1. Read the number of routes that must be closed according to the difficulty.
2. Get all routes.
3. Randomly shuffle the routes.
4. Select routes one by one.
5. Add routes to the game's closed-route set.
6. Stop when the required number of routes has been closed.

5.17 create_game(level)

1. Read the settings for the selected difficulty.
2. Set the starting location to CAMP.
3. Randomly select a hidden treasure location.
4. Initialize energy, maximum energy, moves, hints, score, distance, and wrong-answer count.
5. Create empty sets for used questions and closed routes.

6. Initialize the player's path with CAMP.
 7. Set the ending reason to empty.
 8. Close the required number of initial routes.
 9. Check whether the treasure is still reachable from CAMP.
 10. If not reachable, reopen all initially closed routes.
 11. Return the initialized game.
-

5.18 make_direct_clue(game, routes)

1. Randomly select an open route.
2. Set its destination as the correct answer.
3. Create the answer options from the available routes.
4. Generate the question according to the difficulty level.
5. Create an explanation containing the route information.
6. Return the clue dictionary.

5.19 make_distance_clue(game, routes)

1. If fewer than two routes are available, call make_direct_clue().
2. Otherwise, randomly select a route.
3. Create answer options from the available routes.
4. For Easy, ask for a route with the selected distance.
5. For Medium, find the longest route that is shorter than the selected route.
6. If no shorter route exists, ask for the selected distance directly.
7. For Hard, select the route with the greatest distance.
8. Create the explanation.
9. Return the clue.

5.20 make_map_clue(game, routes)

1. Create possible two-step route pairs.

2. For every available first destination, check its open routes.
 3. Store valid two-step combinations.
 4. If no valid combination exists, return None.
 5. Randomly select a valid pair.
 6. Create the question according to difficulty.
 7. For Hard, calculate the combined energy cost of both routes.
 8. Set the first destination as the correct answer.
 9. Return the clue.
-

5.21 make_clue_id(clue)

1. Read the clue question.
2. Read all answer options.
3. Sort the options.
4. Combine the question and sorted options.
5. Return the resulting string as the clue ID.

5.22 clue_was_recent(clue, game)

1. Generate the clue ID.
2. Check whether it exists in the current game's used-question set.
3. If found, return True.
4. Check the stored questions from recent games.
5. If the clue ID is found, return True.
6. Otherwise, return False.

5.23 remember_clue(clue, game)

1. Generate the clue ID.
2. Add the ID to the current game's used-question set.

5.24 save_game_questions(game)

1. Copy the current game's used-question set.
2. Add it to the recent-game history.
3. If more than five games are stored, remove the oldest game.

5.25 generate_clue(game)

1. Get the open routes from the player's current location.
 2. If no open routes exist, return None.
 3. Try up to 100 times to generate a clue.
 4. Randomly select one of the three clue types.
 5. Generate the selected clue.
 6. Check whether the clue was recently used.
 7. If it is new, remember it and return it.
 8. If no new clue is found after 100 attempts, create a direct clue.
 9. If that clue is also already used, return None.
-

5.26 check_answer(choice, clue, game)

1. Validate whether the selected option number is within the available range.
2. If invalid, display a warning and return False.
3. Retrieve the selected answer.
4. Compare it with the correct answer.
5. If correct:
 - Add the difficulty-specific points.
 - Display the success message.
 - Display the explanation.
 - Return True.
6. If incorrect:
 - Increase the wrong-answer count.

- Reduce energy.
 - Reduce one chance/move.
 - Reduce the score without allowing it below zero.
 - Display the penalties.
7. If energy or moves reaches zero, end the expedition.
 8. Return False.

5.27 give_hint(clue, game)

1. Check whether hints remain.
2. If no hints remain, display a warning and stop.
3. Decrease the hint count by one.
4. Find all incorrect options.
5. Randomly select one incorrect option.
6. Remove that option from the clue.
7. Display the eliminated option.
8. If no incorrect option remains, instruct the player to study the map.

5.28 move_player(game, destination)

1. Find the destination among the player's currently open routes.
2. Retrieve its distance and energy cost.
3. Check whether sufficient energy is available.
4. If insufficient, set the ending reason and return False.
5. Deduct the route's energy cost.
6. Deduct one move.
7. Add the route distance to total distance.
8. Update the current location.
9. Add the destination to the player's path.
10. Display the movement information.

11. Return True.
12. If the route is unavailable, set the ending reason and return False.

5.29 change_routes(game)

1. Generate a random probability value.
 2. Continue only if a route-change event is triggered.
 3. If closed routes exist and the random condition is met, reopen one closed route.
 4. Otherwise, shuffle all routes.
 5. Try closing an open route.
 6. Check whether the treasure remains reachable.
 7. Keep the route closed only if the treasure is still reachable.
 8. Otherwise, reopen the route.
 9. Display the corresponding event message.
-

5.30 play_game(level)

1. Create a new game using the selected difficulty.
2. Display the expedition information.
3. Continue while energy and moves are greater than zero.
4. Check whether the current location contains the treasure.
5. If the treasure is found:
 - Display the victory screen.
 - Display score and distance.
 - Save the used questions.
 - Return to the main menu.
6. Generate a new clue.
7. If no clue is available, end the expedition.
8. Display the clue and its options.

9. Accept commands for:

- M — show map
- S — show status
- H — use hint
- Q — quit
- Number — answer the clue

10. If the answer is correct, use the clue's answer as the destination.

11. Move the player to that destination.

12. After successful movement, dynamically change routes.

13. Repeat until the treasure is found or the expedition ends.

14. Display the loss screen and ending reason when the game ends.

15. Save the game's used questions.

5.31 choose_difficulty()

1. Display Easy, Medium, and Hard options.
2. Read the player's choice.
3. Convert the selected number into a difficulty level.
4. Reject invalid input and ask again.
5. Display the starting resources for the selected level.
6. Wait for the player to continue.
7. Return the selected difficulty.

5.32 show_instructions()

1. Display the game title.
2. Display the starting location.
3. Explain the hidden treasure.
4. Explain clue-based movement.
5. Explain map, hint, and status commands.

6. Explain penalties for wrong answers.
7. Explain resource-based game termination.
8. Explain dynamic route changes.
9. Display the objective of reaching the treasure.

5.33 main()

1. Enable terminal colors.
2. Clear the screen.
3. Display the startup screen.
4. Wait for the player.
5. Display the main menu repeatedly.
6. If the player selects **Start New Expedition**, choose difficulty and start the game.
7. If the player selects **View Map**, display the map.
8. If the player selects **How to Play**, display the instructions.
9. If the player selects **Exit**, display the exit message and terminate.
10. For any other input, display an invalid-choice message and show the menu again.